# Acceso a almacenes de datos

Uso de bases de datos desde servidor

**¿Qué es un ORM?** ..... **2**
¿Qué hace un ORM? ..... 2
Beneficios Principales ..... 3
Ejemplo Conceptual ..... 4
ORMs Populares ..... 4
**Flujo de Acceso a Base de Datos (Java EE/Spring)** ..... **5**
Detalle de las Capas ..... 5
1. Repositorio / DAO (Data Access Object) ..... 5
2. JPA (Java Persistence API) ..... 6
3. Hibernate ..... 6
4. JDBC (Java Database Connectivity) ..... 7
Resumen del Viaje de una Petición ..... 7
**Pasos para crear un proyecto con acceso a base de datos - mysql** ..... **8**
**Configurar driver de base de datos - MySql** ..... **9**
**Configurar la conexión a MySQL** ..... **9**

**Edita el archivo src/main/resources/application.properties:** ..... **9**
# Configuración de la base de datos ..... 10
# JPA / Hibernate ..... 11
**Cómo usar JPA** ..... **13**
El blog - tabla de base de datos ..... 13
Crear una entidad JPA ..... 15
Crear el repositorio: ArticuloRepository.java ..... 22
Crear la capa de servicio: ArticuloService.java ..... 24
Crear el controlador ..... 26
**Estructura general del patrón MVC** ..... **29**
Explicación por capa ..... 29
1. Modelo (Articulo.java) ..... 29
3. Servicio (ArticuloService.java) ..... 30
4. Controlador (ArticuloController.java) ..... 31
5. Vista (Thymeleaf) ..... 31
Resumen global ..... 32
**JpaRepository<T, ID>** ..... **33**
1. Métodos CRUD básicos (de CrudRepository) ..... 33
2. Métodos de paginación y ordenación (de PagingAndSortingRepository) ..... 34
3. Métodos adicionales propios de JpaRepository ..... 34
Además: consultas derivadas (por nombre) ..... 35
**Tabla de anotaciones JPA para clases y campos de entidades** ..... **36**
Notas y buenas prácticas ..... 38

# ¿Qué es un ORM?

Un **ORM** (Object-Relational Mapping, o **Mapeo Objeto-Relacional**) es una técnica de programación que permite **convertir los datos entre el sistema de tipos de un lenguaje de programación orientado a objetos (como Python, Java o C#) y una base de datos relacional**.

## ¿Qué hace un ORM?

En esencia, un ORM actúa como un **traductor** o una capa intermedia.

- Permite a los desarrolladores interactuar con la base de datos **utilizando objetos** y métodos del lenguaje de programación en lugar de escribir consultas SQL directamente.
- El ORM **mapea** las **tablas** de la base de datos a **clases** en el lenguaje de programación, las **filas** a **objetos**, y las **columnas** a **atributos** de esos objetos.

## Beneficios Principales

| Beneficio | Descripción |
| --- | --- |
| **Mayor Productividad** | Reduce la cantidad de código repetitivo de SQL que hay que escribir, permitiendo a los desarrolladores centrarse en la lógica de negocio. |
| **Abstracción de la Base de Datos** | El código se vuelve menos dependiente del motor de base de datos específico (MySQL, PostgreSQL, SQL Server, etc.), facilitando el cambio de uno a otro. |
| **Seguridad** | Muchos ORMs implementan automáticamente mecanismos para prevenir vulnerabilidades comunes como la **inyección SQL**. |
| **Mantenimiento y Legibilidad** | El código escrito en el lenguaje de programación es a menudo más fácil de leer, probar y mantener que las largas cadenas de SQL incrustadas. |

---

## Ejemplo Conceptual

Imagina que tienes una tabla llamada `usuarios` en tu base de datos.

1. **Sin ORM (SQL puro):** Para obtener un usuario, escribirías: `SELECT * FROM usuarios WHERE id = 1;` y luego tendrías que procesar el resultado de la base de datos en tu código.
2. **Con ORM:** Creas una clase `Usuario` y luego llamas a un método del ORM, por ejemplo, en un framework de Python como Django o SQLAlchemy: `Usuario.objects.get(id=1)`. El ORM se encarga de generar y ejecutar el SQL necesario y de devolverte un **objeto `Usuario`**.

---

## ORMs Populares

| Lenguaje | ORMs Comunes |
|---|---|
| **Python** | SQLAlchemy, Django ORM |
| **Java** | Hibernate, JPA (Java Persistence API) |
| **C# (.NET)** | Entity Framework |
| **PHP** | Doctrine, Eloquent (Laravel) |
| **Ruby** | ActiveRecord (Ruby on Rails) |

---

# Flujo de Acceso a Base de Datos (Java EE/Spring)

La relación es una jerarquía donde cada capa superior utiliza y se basa en la capa inmediatamente inferior para funcionar.

1. **Repositorio/DAO** (Capa de Aplicación)
2. **JPA** (Especificación) ⟹
3. **Hibernate** (Implementación) ⟹
4. **JDBC** (Driver/Conexión) ⟹
5. **Base de Datos** (Datos)

---

## Detalle de las Capas

### 1. Repositorio / DAO (Data Access Object)

Esta es la capa más cercana al **código de tu aplicación** (la lógica de negocio).

- **¿Qué es?** Es una clase que define **métodos específicos** para las operaciones de negocio (ej. `encontrarUsuarioPorEmail()`, `guardarNuevoProducto()`).
- **Relación:** Llama a los métodos de la interfaz **JPA** (o de Spring Data JPA, que los implementa automáticamente) para realizar las operaciones.
- **Propósito:** Aislar la lógica de acceso a datos del resto de la aplicación, haciendo que el código sea más limpio y fácil de probar.

---

---

## 2. JPA (Java Persistence API)

Esta es una **especificación** o un conjunto de **interfaces** estándar para el mapeo objeto-relacional en Java.

- **¿Qué es?** Un contrato oficial de Java que define cómo deben ser las clases, las anotaciones (`@Entity`, `@Id`), y los métodos para manejar la persistencia.
- **Relación:** No hace nada por sí misma. Requiere una **implementación** (como Hibernate) para ejecutar las operaciones que le solicita el Repositorio/DAO.
- **Propósito:** Estandarizar el ORM. Si cambias de Hibernate a EclipseLink, el código de tu Repositorio/DAO y tus entidades JPA permanecen igual.

---

## 3. Hibernate

Este es el **proveedor** más popular y una **implementación concreta** de la especificación JPA.

- **¿Qué es?** Una **librería** (el ORM en sí) que toma los objetos de tu aplicación (las entidades JPA) y las peticiones del Repositorio/DAO y las traduce en sentencias **SQL**.
- **Relación:** Utiliza la API **JDBC** para establecer la conexión, enviar el SQL generado y recibir los resultados de la base de datos.
- **Propósito:** Manejar el Mapeo Objeto-Relacional (ORM) automáticamente, incluyendo la gestión de transacciones, caché y generación de SQL optimizado.

---

---

### 4. JDBC (Java Database Connectivity)

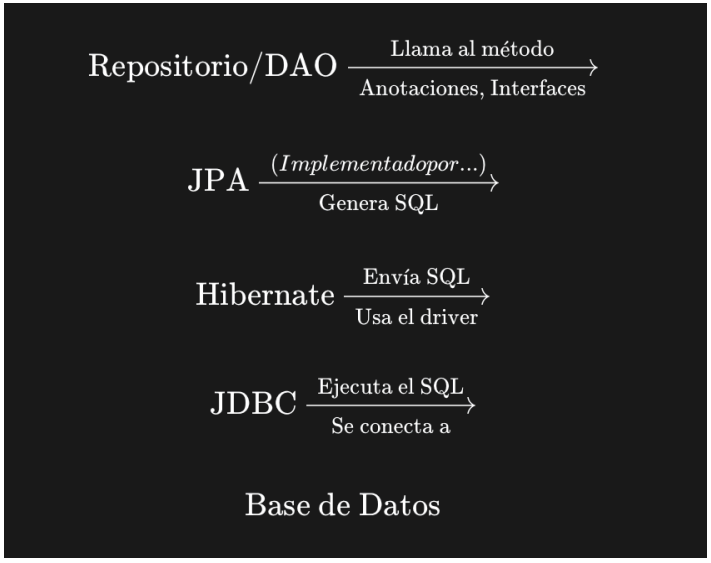
Esta es la capa de más **bajo nivel** en el lado de Java.

- **¿Qué es?** Es una **API** (conjunto de clases e interfaces) estándar de Java que permite a las aplicaciones conectarse directamente a una base de datos.
- **Relación: Hibernate** (u otros ORMs) usa JDBC para comunicarse con el driver específico de la base de datos (ej. un driver para PostgreSQL o MySQL).
- **Propósito:** Proporcionar una forma **universal** de interactuar con diferentes bases de datos. Es lo que realmente ejecuta el SQL.

---

## Resumen del Viaje de una Petición

Cuando llamas a un método en tu **Repositorio/DAO**:



# Pasos para crear un proyecto con acceso a base de datos - mysql

- Crear el proyecto
- Configurar driver de la base de datos MySql
- Configurar la conexión a la base de datos
- Por cada funcionalidad/tabla
    - Crear la entidad
    - Crear el repositorio
    - Crear el servicio
    - Crear el controlador y vistas

# Configurar driver de base de datos - MySql

Añadir a **pom.xml** al inicio de `<dependencies>`:

```
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
```

# Configurar la conexión a MySQL

Edita el archivo `src/main/resources/application.properties`:

```
# ================================
# Configuración de la base de datos
# ================================
spring.datasource.url=jdbc:mysql://localhost:3306/nombre de la base de datos
spring.datasource.username=usuario
spring.datasource.password=contraseña
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# ================================
# JPA / Hibernate
# ================================
```

---

```
spring.jpa.hibernate.ddl-auto=none
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
spring.jpa.properties.hibernate.format_sql=true
```

---

# # Configuración de la base de datos

**a.- spring.datasource.url**

Es la **URL de conexión JDBC** que indica:

- `jdbc:mysql://` → protocolo JDBC y tipo de base de datos (MySQL).
- `localhost` → el servidor donde está MySQL (puede ser una IP o dominio).
- `3306` → el puerto por defecto de MySQL.
- el nombre de la base de datos.

**b.- spring.datasource.username**

Usuario con el que Spring Boot se conectará a MySQL.

**c.- spring.datasource.password**

Contraseña asociada al usuario anterior (`root` o el que uses).

**d.- spring.datasource.driver-class-name**

Nombre completo de la clase **driver JDBC** que permite que Java se comunique con MySQL.

- `com.mysql.cj.jdbc.Driver` pertenece al **conector oficial de MySQL (Connector/J)**.
- Normalmente Spring Boot lo detecta automáticamente, pero es buena práctica indicarlo explícitamente.

# # JPA / Hibernate

**a.- spring.jpa.hibernate.ddl-auto=none**

**Significado:**
Indica qué debe hacer Hibernate con el **esquema de la base de datos** (las tablas).

**Valores posibles:**

| Valor | Significado |
|---|---|
| `none` | No modifica la base de datos (la tabla ya existe). |
| `validate` | Comprueba que las tablas existen y coinciden con las entidades, pero no cambia nada. |
| `update` | Crea o modifica automáticamente las tablas según las entidades. |
| `create` | Borra todas las tablas y las vuelve a crear cada vez que arrancas. |
| `create-drop` | Igual que `create`, pero borra las tablas al detener la app. |

---

**b.- spring.jpa.show-sql=true**

**Significado:**
Muestra por consola las **consultas SQL** que Hibernate ejecuta.
Es útil para depurar o entender qué operaciones se están realizando sobre la base de datos.

**c.- spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect**

**Significado:**
Indica a Hibernate qué **dialecto SQL** debe usar, es decir, qué tipo de SQL generar según la base de datos.

- Cada SGBD (MySQL, PostgreSQL, Oracle, etc.) tiene su propio dialecto.
- Con `MySQLDialect`, Hibernate generará sentencias adaptadas a MySQL (por ejemplo, `AUTO_INCREMENT`).

**d.-spring.jpa.properties.hibernate.format_sql=true**

**Significado:**

Hace que las sentencias SQL que aparecen en consola estén **formateadas y legibles** (saltos de línea e indentación).

---

# Cómo usar JPA.

El sitio oficial donde puedes consultar la documentación de Spring Data JPA (usado con Spring Boot) es:

**Spring Data JPA – Reference Documentation**
https://docs.spring.io/spring-data/jpa/reference/index.html docs.spring.io

Además, un buen punto de partida práctico es el tutorial “Accessing Data with JPA” de Spring Guides:
https://spring.io/guides/gs/accessing-data-jpa

Esta es de JPA con mysql:
https://spring.io/guides/gs/accessing-data-mysql

---

## El blog - tabla de base de datos

El ejemplo que se va a seguir es el de un sitio para publicar artículos, es decir, un blog.

La tabla que se va a utilizar es:

```
CREATE TABLE articulos (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,      -- Identificador único del artículo
  titulo VARCHAR(200) NOT NULL,              -- Título del artículo
  contenido TEXT NOT NULL,                   -- Cuerpo del artículo
  autor VARCHAR(100) NOT NULL,               -- Nombre del autor
  fecha_publicacion DATE NOT NULL,           -- Fecha en que se publicó
  publicado BOOLEAN DEFAULT FALSE NOT NULL   -- Indica si está publicado o en borrador
);
```

```
-- Datos de ejemplo
INSERT INTO articulos (titulo, contenido, autor, fecha_publicacion, publicado) VALUES
('Introducción a Spring Boot',
 'Spring Boot simplifica el desarrollo de aplicaciones Java al eliminar gran parte de la configuración necesaria...',
 'Ana García',
 '2025-01-10',
 TRUE),
('Buenas prácticas con JPA',
 'En este artículo veremos cómo estructurar las entidades y los repositorios para mantener un código limpio...',
 'Luis Pérez',
 '2025-02-05',
 TRUE),
('Cómo usar H2 para pruebas rápidas',
 'H2 es una base de datos en memoria muy útil para pruebas y demostraciones con Spring Boot...',
 'Marta Santos',
 '2025-02-25',
 FALSE);
```

**Para el ejemplo se crea una base de datos y un usuario con permisos con esa base de datos**

## Crear una entidad JPA

Después mapearemos esta tabla con una clase `Articulo` en Java con los campos:

| Columna SQL | Tipo Java sugerido | Anotaciones clave |
|---|---|---|
| id | `Long` | `@Id`, `@GeneratedValue(strategy = GenerationType.IDENTITY)` |
| titulo | `String` | `@Column(nullable = false)` |
| contenido | `String` | `@Lob`, `@Column(nullable = false)` |
| autor | `String` | `@Column(nullable = false)` |
| fecha_publicacion | `LocalDate` | `@Column(nullable = false)` |
| publicado | `boolean` | `@Column(nullable = false)` |

El código puede ser como este:

```
/**
 * Clase de entidad que representa la tabla 'articulos' en la base de datos.
 * Cada instancia de esta clase corresponde a una fila de la tabla.
 */
@Entity                         // Marca la clase como una entidad JPA.
@Table(name = "articulos")      // Indica el nombre exacto de la tabla en la base de datos.
public class Articulo {

    // ============================
    // Atributos (columnas)
    // ============================

    /**
     * Campo 'id' -> Clave primaria, generada automáticamente.
     */
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Equivale a AUTO_INCREMENT en MySQL.
    private Long id;

    /**
     * Campo 'titulo' -> Título del artículo.
     */
    @Column(nullable = false, length = 200)
    private String titulo;

    /**
```

```
     * Campo 'contenido' -> Texto completo del artículo.
     * Se marca como @Lob (Large Object) para permitir textos largos.
     */
    @Lob
    @Column(nullable = false)
    private String contenido;

    /**
     * Campo 'autor' -> Nombre del autor que escribió el artículo.
     */
    @Column(nullable = false, length = 100)
    private String autor;

    /**
     * Campo 'fecha_publicacion' -> Fecha en que se publicó el artículo.
     * Se mapea con LocalDate para trabajar con fechas sin horas.
     */
    @Column(name = "fecha_publicacion", nullable = false)
    private LocalDate fechaPublicacion;

    /**
     * Campo 'publicado' -> Indica si el artículo está visible o en borrador.
     */
    @Column(nullable = false)
    private boolean publicado;

    // ============================
    // Constructores
```

```
    // ============================

    /** Constructor vacío obligatorio para JPA. */
    public Articulo() {
    }

    /** Constructor con parámetros para crear instancias fácilmente. */
    public Articulo(String titulo, String contenido, String autor, LocalDate fechaPublicacion, boolean
publicado) {
        this.titulo = titulo;
        this.contenido = contenido;
        this.autor = autor;
        this.fechaPublicacion = fechaPublicacion;
        this.publicado = publicado;
    }

    // ============================
    // Métodos getter y setter
    // ============================

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }
```

```
    public String getTitulo() {
        return titulo;
    }

    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }

    public String getContenido() {
        return contenido;
    }

    public void setContenido(String contenido) {
        this.contenido = contenido;
    }

    public String getAutor() {
        return autor;
    }

    public void setAutor(String autor) {
        this.autor = autor;
    }

    public LocalDate getFechaPublicacion() {
        return fechaPublicacion;
    }
```

```
    public void setFechaPublicacion(LocalDate fechaPublicacion) {
        this.fechaPublicacion = fechaPublicacion;
    }

    public boolean isPublicado() {
        return publicado;
    }

    public void setPublicado(boolean publicado) {
        this.publicado = publicado;
    }

    // ============================
    // Método toString (para depuración)
    // ============================

    @Override
    public String toString() {
        return "Articulo{" +
                "id=" + id +
                ", titulo='" + titulo + '\'' +
                ", autor='" + autor + '\'' +
                ", fechaPublicacion=" + fechaPublicacion +
                ", publicado=" + publicado +
                '}';
    }
}
```

---

Explicación de las anotaciones

| Anotación | Qué hace |
|---|---|
| `@Entity` | Marca la clase como entidad JPA (mapeada a una tabla). |
| `@Table(name = "articulos")` | Indica el nombre de la tabla en la base de datos. |
| `@Id` | Define la clave primaria. |
| `@GeneratedValue(strategy = GenerationType.IDENTITY)` | Usa auto-incremento de MySQL. |
| `@Column(...)` | Configura restricciones o nombres de columnas. |
| `@Lob` | Permite almacenar textos largos (`TEXT` en MySQL). |
| `LocalDate` | Se mapea automáticamente a `DATE` en SQL. |

## Crear el repositorio: ArticuloRepository.java

```
/**
 * Interfaz de repositorio para la entidad Articulo.
 *
 * Spring Data JPA creará automáticamente la implementación en tiempo de ejecución.
 *
 * Hereda de JpaRepository, que ya proporciona métodos CRUD básicos:
 * - findAll() -> devuelve todos los artículos
 * - findById(id) -> busca un artículo por su id
 * - save(obj) -> inserta o actualiza un artículo
 * - deleteById(id) -> elimina un artículo
 */
@Repository
public interface ArticuloRepository extends JpaRepository<Articulo, Long> {
    // ============================
    // Consultas personalizadas
    // ============================

    /**
     * Encuentra todos los artículos de un autor concreto.
     * El nombre del método sigue la convención de Spring Data JPA.
     */
    List<Articulo> findByAutor(String autor);

    /**
     * Devuelve los artículos que estén publicados (true) o no (false).
     */
    List<Articulo> findByPublicado(boolean publicado);
```

---

```
    /**
     * Busca artículos cuyo título contenga una palabra clave (ignorando mayúsculas/minúsculas).
     */
    List<Articulo> findByTituloContainingIgnoreCase(String palabraClave);
}
```

---

**Explicación de lo que hace Spring Data JPA aquí**

- `JpaRepository<Articulo, Long>`
  Indica que este repositorio trabaja con la entidad `Articulo` y su clave primaria es de tipo `Long`.
- **Spring Data JPA genera automáticamente todos los métodos básicos** (sin código SQL):
  - `findAll()` → SELECT * FROM articulos
  - `findById(…)` → SELECT * FROM articulos WHERE id = ?
  - `save(…)` → INSERT o UPDATE según el caso
  - `deleteById(…)` → DELETE FROM articulos WHERE id = ?

- **Consultas derivadas por nombre de método**
  Spring analiza el nombre del método y construye la consulta automáticamente.

  Por ejemplo:

  ```
  findByAutor("Ana García")
  ```

  Se convierte en:

  ```
  SELECT * FROM articulos WHERE autor = 'Ana García';
  ```

## Crear la capa de servicio: `ArticuloService.java`

```
/**
 * Capa de servicio que actúa como intermediaria entre el controlador
 * y el repositorio. Aquí se aplicaría la lógica de negocio si fuera necesaria.
 */
@Service
public class ArticuloService {

    // Inyección automática del repositorio (Spring lo gestiona).
    private final ArticuloRepository articuloRepository;

    // Constructor para inyección de dependencias.
    public ArticuloService(ArticuloRepository articuloRepository) {
        this.articuloRepository = articuloRepository;
    }

    // ============================
    // Métodos CRUD
    // ============================

    /** Obtiene todos los artículos de la base de datos. */
    public List<Articulo> listarTodos() {
        return articuloRepository.findAll();
    }

    /** Busca un artículo por su ID. */
    public Optional<Articulo> buscarPorId(Long id) {
        return articuloRepository.findById(id);
    }

    /** Guarda (inserta o actualiza) un artículo. */
```

```
    public Articulo guardar(Articulo articulo) {
        return articuloRepository.save(articulo);
    }

    /** Elimina un artículo por su ID. */
    public void eliminar(Long id) {
        articuloRepository.deleteById(id);
    }

    // ============================
    // Métodos personalizados
    // ============================

    /** Devuelve los artículos de un autor específico. */
    public List<Articulo> buscarPorAutor(String autor) {
        return articuloRepository.findByAutor(autor);
    }

    /** Devuelve los artículos según su estado de publicación. */
    public List<Articulo> buscarPorPublicado(boolean publicado) {
        return articuloRepository.findByPublicado(publicado);
    }

    /** Busca artículos que contengan una palabra en el título. */
    public List<Articulo> buscarPorPalabraClave(String palabra) {
        return articuloRepository.findByTituloContainingIgnoreCase(palabra);
    }
}
```

# Crear el controlador

```
/**
 * Controlador MVC (no REST) que maneja las peticiones y devuelve vistas Thymeleaf.
 * Se apoya en ArticuloService para la lógica de negocio y acceso a datos.
 */
@Controller
@RequestMapping("/articulos")
public class ArticuloController {

    private final ArticuloService articuloService;

    public ArticuloController(ArticuloService articuloService) {
        this.articuloService = articuloService;
    }

    // ============================
    // Mostrar todos los artículos
    // ============================
    @GetMapping
    public String listarArticulos(Model model) {
        model.addAttribute("listaArticulos", articuloService.listarTodos());
        return "lista-articulos"; // plantilla Thymeleaf: lista-articulos.html
    }

    // ============================
    // Mostrar formulario para crear nuevo artículo
    // ============================
    @GetMapping("/nuevo")
    public String mostrarFormularioNuevo(Model model) {
        model.addAttribute("articulo", new Articulo());
        return "form-articulo"; // plantilla: form-articulo.html
    }

    // ============================
    // Guardar nuevo artículo
    // ============================
```

```
@PostMapping("/guardar")
public String guardarArticulo(@ModelAttribute("articulo") Articulo articulo) {
    // Si no se ha especificado fecha, la ponemos automáticamente
    if (articulo.getFechaPublicacion() == null) {
        articulo.setFechaPublicacion(LocalDate.now());
    }
    articuloService.guardar(articulo);
    return "redirect:/articulos";
}

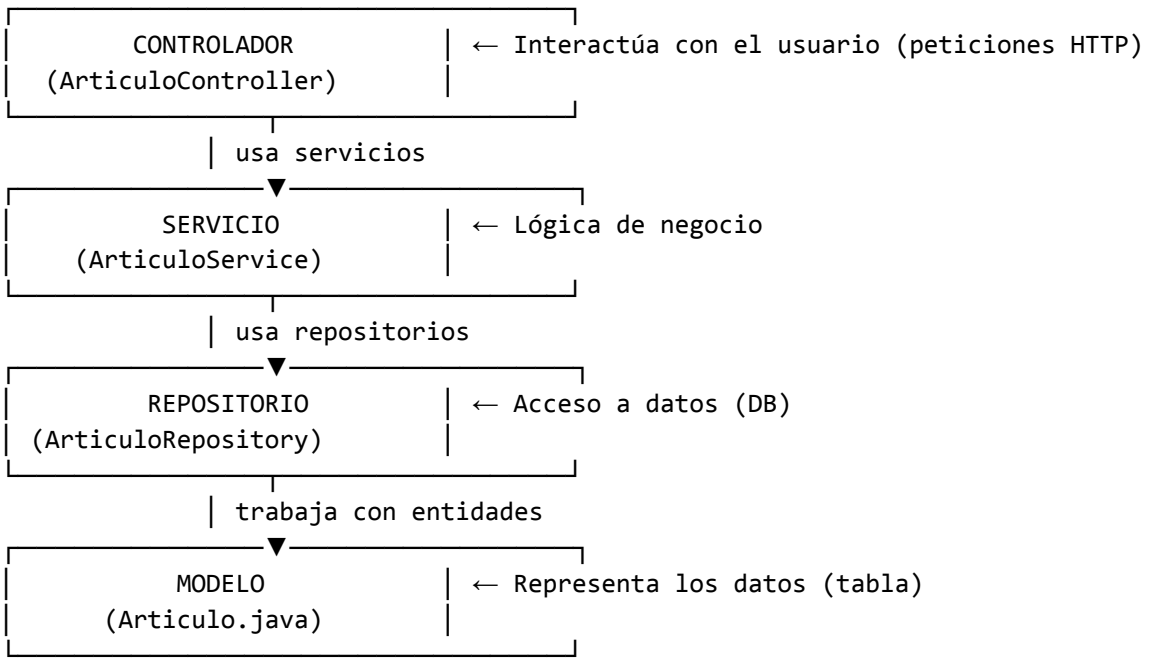
// ============================
// Mostrar formulario de edición
// ============================
@GetMapping("/editar/{id}")
public String mostrarFormularioEditar(@PathVariable Long id, Model model) {
    Articulo articulo = articuloService.buscarPorId(id)
            .orElseThrow(() -> new IllegalArgumentException("ID de artículo no válido: " + id));
    model.addAttribute("articulo", articulo);
    return "form-articulo"; // reutiliza la misma plantilla de formulario
}

// ============================
// Eliminar artículo
// ============================
@GetMapping("/eliminar/{id}")
public String eliminarArticulo(@PathVariable Long id) {
    articuloService.eliminar(id);
    return "redirect:/articulos";
}

// ============================
// Ver detalle de un artículo
// ============================
@GetMapping("/{id}")
public String verDetalle(@PathVariable Long id, Model model) {
    Articulo articulo = articuloService.buscarPorId(id)
            .orElseThrow(() -> new IllegalArgumentException("ID de artículo no válido: " + id));
```

```
        model.addAttribute("articulo", articulo);
        return "detalle-articulo"; // plantilla: detalle-articulo.html
    }
}
```

# Estructura general del patrón MVC




## Explicación por capa

### 1. Modelo (`Articulo.java`)

- Es la **entidad JPA** que representa la **tabla de la base de datos** (`articulos`).
- Define los **atributos** (columnas) y cómo se mapean a la base de datos.
- Ejemplo: `titulo`, `contenido`, `autor`, etc.

---

- Pertenece a la **capa de datos**.

**Ubicación:** `com.ejemplo.blog.modelo.entidades`
**Función en MVC: Modelo**

---

## 2. Repositorio (`ArticuloRepository.java`)

- Es la **interfaz de acceso a datos**.
- Comunica directamente la aplicación con la base de datos mediante **JPA/Hibernate**.
- No contiene lógica, solo define consultas (por convención o personalizadas).
- Spring Data crea la implementación automáticamente.

**Ubicación:** `com.ejemplo.blog.modelo.repositorios`
**Función en MVC:** Parte del **Modelo** (capa de persistencia)

---

## 3. Servicio (`ArticuloService.java`)

- Contiene la **lógica de negocio** o "reglas del dominio".
- Aquí se toman decisiones como:
    - qué hacer antes o después de guardar un artículo,
    - validaciones,
    - cálculos o transformaciones.
- Llama al repositorio, pero no accede directamente a la base de datos.

---

**Ubicación:** `com.ejemplo.blog.servicios`
**Función en MVC: Capa intermedia entre Modelo y Controlador**
(a veces se la llama **"Capa de negocio"**, no estrictamente parte de MVC, pero muy habitual en arquitecturas limpias con Spring).

---

## 4. Controlador (`ArticuloController.java`)

- Es la **C del MVC**.
- Recibe las peticiones HTTP del usuario y decide qué hacer:
  - si debe mostrar una vista Thymeleaf,
  - si debe guardar un artículo,
  - si debe eliminar o listar.
- Llama al **servicio** (no al repositorio directamente).
- Devuelve el nombre de la **vista Thymeleaf** que se debe renderizar.

**Ubicación:** `com.ejemplo.blog.controladores`
**Función en MVC: Controlador**

---

## 5. Vista (Thymeleaf)

- Los archivos `.html` que representan lo que el usuario ve en el navegador.
- Se guardan en: `src/main/resources/templates/`
- Thymeleaf combina el HTML con datos del modelo (`Model` en el controlador).

**Función en MVC: Vista**

# Resumen global

| Capa | Clase(s) | Rol |
|---|---|---|
| **Modelo (Datos)** | `Articulo.java`, `ArticuloRepository.java` | Representan la información y su acceso. |
| **Servicio (Lógica)** | `ArticuloService.java` | Contiene la lógica de negocio y coordina el flujo entre modelo y controlador. |
| **Controlador (Presentación)** | `ArticuloController.java` | Recibe peticiones del usuario, usa el servicio y devuelve vistas. |
| **Vista (Interfaz)** | `Plantillas Thymeleaf (.html)` | Muestran los datos al usuario. |

# JpaRepository<T, ID>

`JpaRepository` es una **interfaz genérica** que extiende a:

- `CrudRepository<T, ID>` → operaciones CRUD básicas
- `PagingAndSortingRepository<T, ID>` → operaciones con paginación y ordenación

Por eso hereda **todos los métodos** de ambas, además de algunos adicionales.

## 1. Métodos CRUD básicos (de `CrudRepository`)

| Método | Descripción | Ejemplo |
|---|---|---|
| `List<T> findAll()` | Devuelve todos los registros. | `findAll()` |
| `Optional<T> findById(ID id)` | Busca un registro por su ID. | `findById(1L)` |
| `T save(T entity)` | Inserta o actualiza una entidad. | `save(articulo)` |
| `void delete(T entity)` | Elimina una entidad. | `delete(articulo)` |
| `void deleteById(ID id)` | Elimina por ID. | `deleteById(3L)` |
| `boolean existsById(ID id)` | Verifica si existe un registro con ese ID. | `existsById(1L)` |
| `long count()` | Devuelve la cantidad total de registros. | `count()` |

## 2. Métodos de paginación y ordenación (de PagingAndSortingRepository)

| Método | Descripción | Ejemplo |
|---|---|---|
| `List<T> findAll(Sort sort)` | Devuelve todos los registros ordenados. | `findAll(Sort.by("titulo"))` |
| `Page<T> findAll(Pageable pageable)` | Devuelve una página de resultados. | `findAll(PageRequest.of(0, 10))` |

## 3. Métodos adicionales propios de JpaRepository

| Método | Descripción | Ejemplo |
|---|---|---|
| `void flush()` | Fuerza la escritura de los cambios pendientes en la base de datos. | `flush()` |
| `T saveAndFlush(T entity)` | Guarda y hace `flush()` inmediatamente. | `saveAndFlush(articulo)` |
| `List<T> saveAllAndFlush(Iterable<T> entities)` | Guarda varias entidades y hace `flush()`. | `saveAllAndFlush(listaArticulos)` |

| | | |
|---|---|---|
| `void` **`deleteAllInBatch`**`()` | Elimina todos los registros en un solo lote (sin cargar entidades). | `deleteAllInBatch()` |
| `void` **`deleteAllInBatch`**`(Iterable<T> entities)` | Elimina una colección de entidades en lote. | `deleteAllInBatch(listaArticulos)` |
| `T getOne(ID id)` *(deprecated, usar `getReferenceById()`)* | Devuelve una referencia “perezosa” al objeto (sin cargarlo completo). | `getReferenceById(5L)` |
| `T` **`getReferenceById`**`(ID id)` | Igual que `getOne`, versión actualizada. | `getReferenceById(5L)` |

## Además: consultas derivadas (por nombre)

Spring Data JPA genera consultas automáticas si defines métodos con nombres adecuados:

| Método personalizado | Consulta generada |
|---|---|
| `findByAutor(String autor)` | `SELECT * FROM articulos WHERE autor = ?` |
| `findByPublicado(boolean publicado)` | `SELECT * FROM articulos WHERE publicado = ?` |
| `findByTituloContaining(String texto)` | `SELECT * FROM articulos WHERE titulo LIKE %texto%` |

# Tabla de anotaciones JPA para clases y campos de entidades

| Anotación | Se aplica a | Descripción / Función | Ejemplo |
|---|---|---|---|
| `@Entity` | Clase | Indica que la clase es una **entidad JPA** (se mapea a una tabla). | `@Entity public class Articulo { ... }` |
| `@Table(name = "nombre_tabla")` | Clase | Especifica el nombre de la tabla (opcional si coincide con el nombre de la clase). | `@Table(name = "articulos")` |
| `@Id` | Campo | Marca el atributo como **clave primaria**. | `@Id private Long id;` |
| `@GeneratedValue(strategy = …)` | Campo | Define cómo se genera el valor del ID. Estrategias: `AUTO`, `IDENTITY`, `SEQUENCE`, `TABLE`. | `@GeneratedValue(strategy = GenerationType.IDENTITY)` |
| `@Column(name = "columna", nullable = …, unique = …, length = …)` | Campo | Configura la **columna** (nombre, longitud, si puede ser nula, si es única, etc.). | `@Column(nullable = false, length = 100)` |
| `@Lob` | Campo | Indica que el atributo es un **objeto grande** (texto largo o binario). Se usa con `TEXT` o `BLOB`. | `@Lob private String contenido;` |
| `@Transient` | Campo | Excluye el atributo del mapeo; **no se guarda** en la base de datos. | `@Transient private int contadorTemporal;` |

| | | | |
|---|---|---|---|
| **`@Temporal(TemporalType.DATE/TIME/TIMESTAMP)`** | Campo | (Para `java.util.Date` o `Calendar`) Define el tipo de dato temporal que se guarda. *(Con `LocalDate` o `LocalDateTime` no se necesita)* | `@Temporal(TemporalType.DATE)` |
| **`@Enumerated(EnumType.STRING)`** | Campo (enum) | Guarda un valor enumerado como texto (`STRING`) o número (`ORDINAL`). | `@Enumerated(EnumType.STRING)` |
| **`@ManyToOne`** | Campo (relación) | Define una relación **muchos a uno** (varios artículos → un autor). | `@ManyToOne private Autor autor;` |
| **`@OneToMany(mappedBy = …)`** | Campo (relación) | Define una relación **uno a muchos** (un autor → varios artículos). | `@OneToMany(mappedBy = "autor")` |
| **`@OneToOne`** | Campo (relación) | Define una relación **uno a uno**. | `@OneToOne private Imagen imagen;` |
| **`@ManyToMany`** | Campo (relación) | Define una relación **muchos a muchos** (por ejemplo, artículos y etiquetas). | `@ManyToMany private List<Etiqueta> etiquetas;` |
| **`@JoinColumn(name = "columna_id")`** | Campo (relación) | Indica la **columna de clave foránea** en una relación. | `@JoinColumn(name = "autor_id")` |
| **`@JoinTable(name = "tabla_intermedia", joinColumns = …, inverseJoinColumns = …)`** | Campo (relación M:N) | Define la **tabla de unión** en relaciones muchos-a-muchos. | `@JoinTable(name="articulos_etiquetas")` |
| **`@Embedded`** | Campo | Inserta otra clase dentro de la tabla (campos embebidos). | `@Embedded private Direccion direccion;` |
| **`@Embeddable`** | Clase | Marca una clase que puede ser embebida en otra entidad. | `@Embeddable public class Direccion { … }` |

| | | | |
|---|---|---|---|
| **`@Version`** | Campo | Se usa para **control de concurrencia optimista** (gestiona versiones). | `@Version private int version;` |
| **`@CreationTimestamp`** *(Hibernate)* | Campo | Asigna automáticamente la fecha/hora de creación. | `@CreationTimestamp private LocalDateTime creado;` |
| **`@UpdateTimestamp`** *(Hibernate)* | Campo | Asigna automáticamente la fecha/hora de última actualización. | `@UpdateTimestamp private LocalDateTime actualizado;` |

## Notas y buenas prácticas

- Usa `LocalDate` / `LocalDateTime` en lugar de `java.util.Date` (más moderno y compatible con Hibernate 6 / JPA 3).
- No declares campos estáticos (`static`) ni constantes como parte del modelo.
- Cada entidad **debe tener un constructor vacío** (requerido por JPA).
- Puedes combinar `@Data` de **Lombok** (si lo usas) para generar automáticamente getters y setters.
- Si tienes relaciones entre entidades, especifica bien el tipo de carga (`fetch = FetchType.LAZY` o `EAGER`) para optimizar el rendimiento.